



TP2 Entrées-sorties sur fichiers et traitement d'images en vhdL

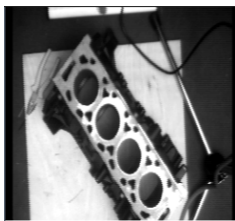
1 Séance

Les buts de ce TP sont :

- L'utilisation des sous programmes (fonctions et procédures),
- Les tableaux à 2 dimensions non contraints,
- L'utilisation du package Textio,
- L'utilisation des entrées sorties avec des fichiers.

1 – Préparation de l'environnement :

Créer un nouveau répertoire pour le TP2 dans lequel vous allez copier le contenu du répertoire « Supports TP2 » présent sur Moodle.



Parmi les fichiers du répertoire supports tp2 il y a :

- 3 fichiers image qu'on voudrait traiter (format gdr).
- Un fichier package contenant une fonction pour extraire les pixels d'une image (fichier) vers un tableau 2D et une procédure pour mettre le tableau des pixels traités dans un fichier image.
- 3 Utilitaires pour visualiser les images (xv) ainsi que la conversion de fichiers images format gdr vers des fichiers texte (gdr2txt) et vice versa (txt2gdr). Ces utilitaires sont à utiliser sous une fenêtre invité commande de windows (cmd.exe).

Visualisez les trois images en utilisant xv.

Convertissez les 3 images gdr au formats txt en suivant la syntaxe suivante (*sous une fenêtre « invité Commade »*) :

`gdr2txt xxxx.gdr xxxx .txt` (*pour convertir fichier d'une image format gdr vers un fichier texte*)

Les fichiers à convertir doivent être au même endroit où se trouvent les exécutables des utilitaires (xv, gdr2txt et txt2gdr).

Une fois les fichiers au format txt sont créées à la suite des traitements, il faudra les convertir au format gdr en utilisant la syntaxe suivante :

`txt2gdr xxxxyyy.txt xxxxyyy.gdr 256 256` (*pour convertir un fichier texte vers un fichier d'une image au format gdr*)

Conseil : Mettez les images dans le même répertoire que votre projet Modelsim

2-Les traitements à faire :

Effectuer les traitements suivants sur les 3 images fournies puis sur les images à synthétiser montrées sur la page suivante.

- 1 – Une Inversion simple (Voir exemple dans poly du cours)
- 2 – Moyenne sur un masque de 3x3
- 3 – Le gradient horizontal G_x et le gradient vertical G_y en utilisant selon le masque de Sobel.
(voir masques page suivante)

Il est intéressant de visualiser les gradients horizontaux et verticaux avant de calculer le gradient. C'est à dire générer les images correspondantes puis les observer avec l'outil xv. Utiliser l'opérateur *abs* (valeur absolue) lors du calcul G_x et G_y .

- 4 – Le gradient $G = \sqrt{G_x^2 + G_y^2}$ (Utiliser la fonction sqrt présente dans le package math_real de la bibliothèque ieeec (voir prototype dans C:\PROGRAMS\altera\13.0sp1\quartus\libraries\vhdl\ieeec)

- 5 – Question : Réponse à mettre dans le compte rendu : Que doit – on faire si l'on veut synthétiser ces traitements ?



$$G_x = \begin{pmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{pmatrix}$$

$$G_y = \begin{pmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix}$$

3 - Compte rendu :

Le compte rendu doit inclure :

- Les fichiers sources,
- **Un seul fichier pdf dans lequel vous mettez toutes vos images traitées correctement nommées (type de traitement).**

Le tout dans une archive (de préférence .zip) nommée comme suit : **TP2_VHDL_nom1_nom2.zip** à déposer sur Moodle à la fin de la séance.

TP2 : Traitement images en VHDL

Les objectifs de ce TP sont :

- L'utilisation des sous programmes (fonctions et procédures),
- Les tableaux à 2 dimensions non contraints,
- L'utilisation du package Textio,
- L'utilisation des entrées sorties avec des fichiers.

TP2 : Traitement images en VHDL

Environnement :

3 images (format gdr)

1 package (entree_sortie_pkg):

- Type de données (tableau 2D)
- 2 sous programmes
- Getpixels : extraction des pixels d'un fichier image → tableau 2D
- Putpixels : Tableau 2D (pixels traités) → fichier image

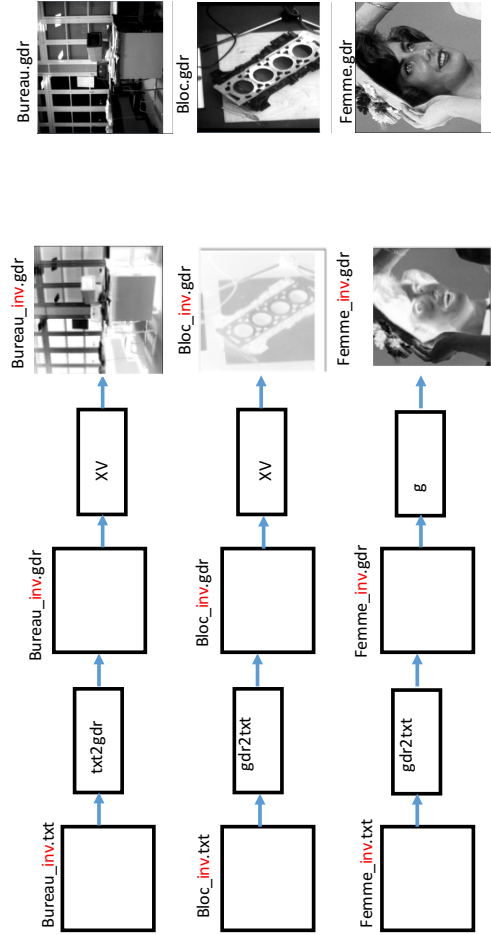
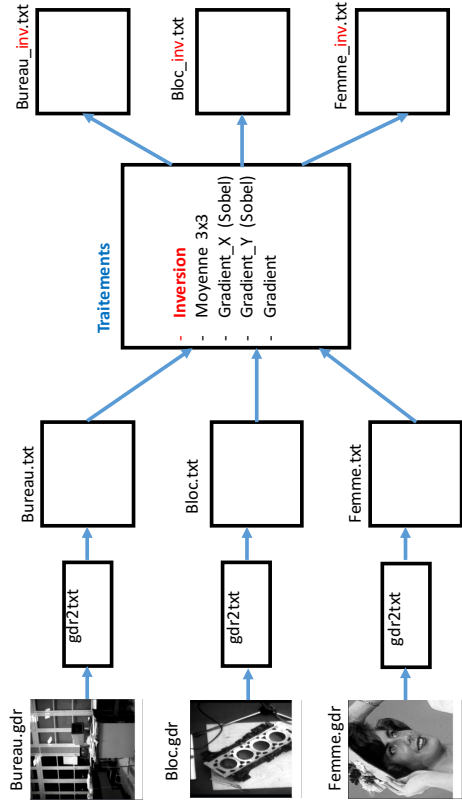
3 utilitaires :

- gdr2txt (conversion d'un fichier image gdr → fichier image txt
- txt2gdr (conversion d'un fichier image txt → fichier image gdr

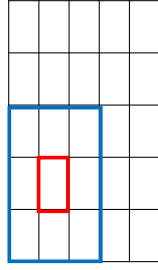
A exécuter
Sous invité commandes
(DOS)

- XV visualisation des images gdr

→ Fournis sur Moodle



La moyenne : Masque de 3x3



Somme des 9 pixels / 9

Le gradient : Masque de 3x3



→ Utiliser valeur absolue

Le gradient $G = \sqrt{G_x^2 + G_y^2}$
(Utiliser la fonction sqrt présente dans le package math_real de la bibliothèque ieee (voir prototype dans C:\PROGRAMS\altera\13.0sp1\quartus\libraries\vhdl\ieee))

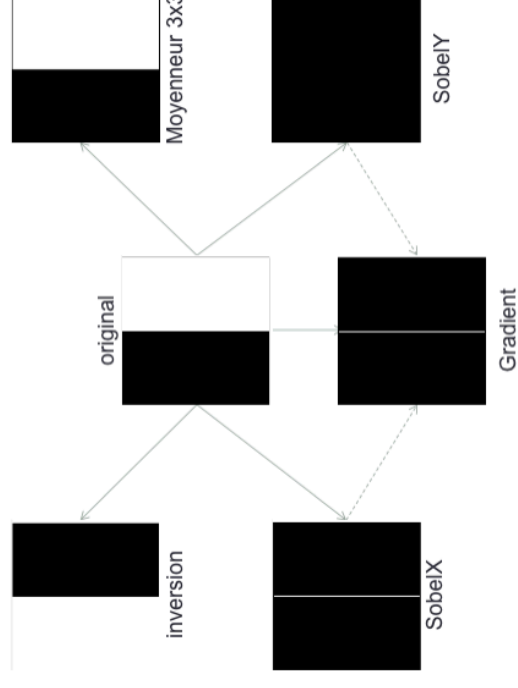
Synthétiser les images :

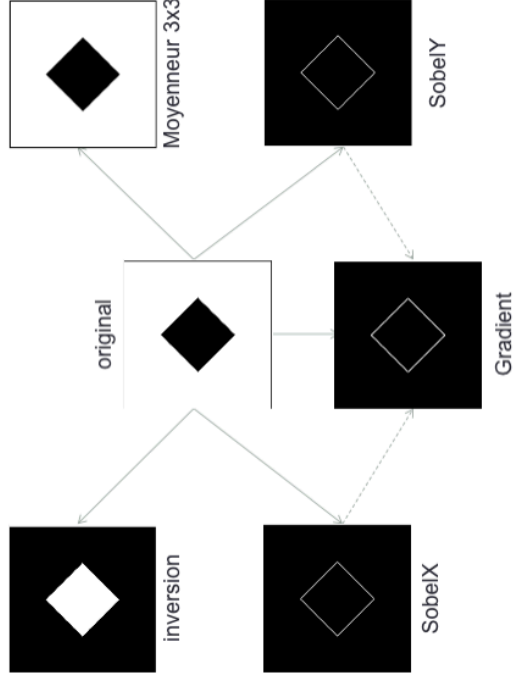
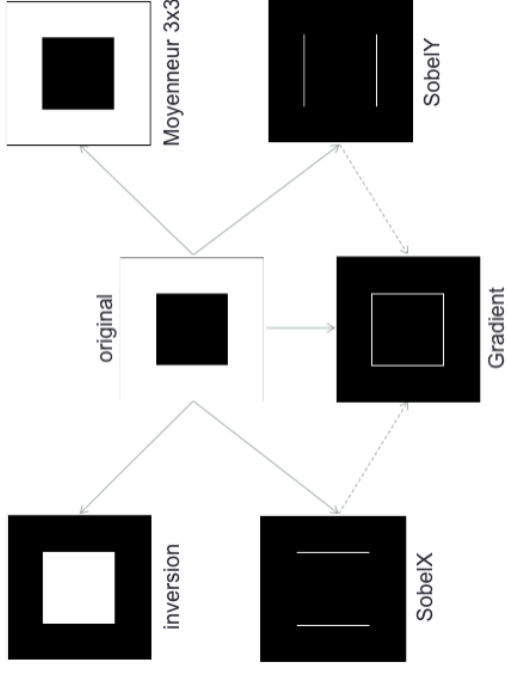
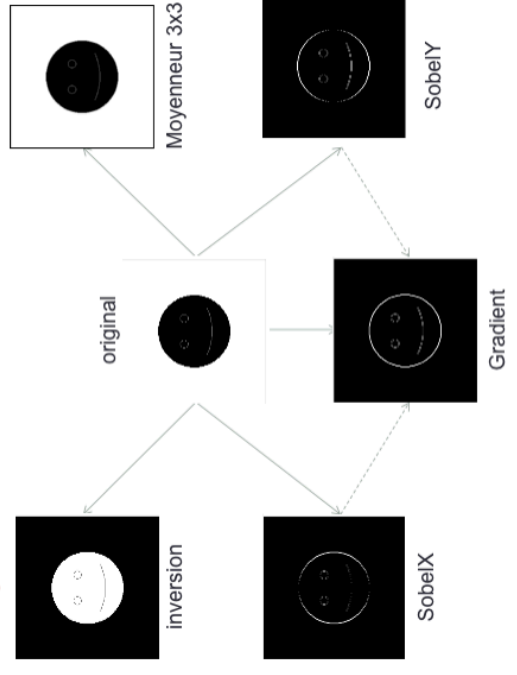
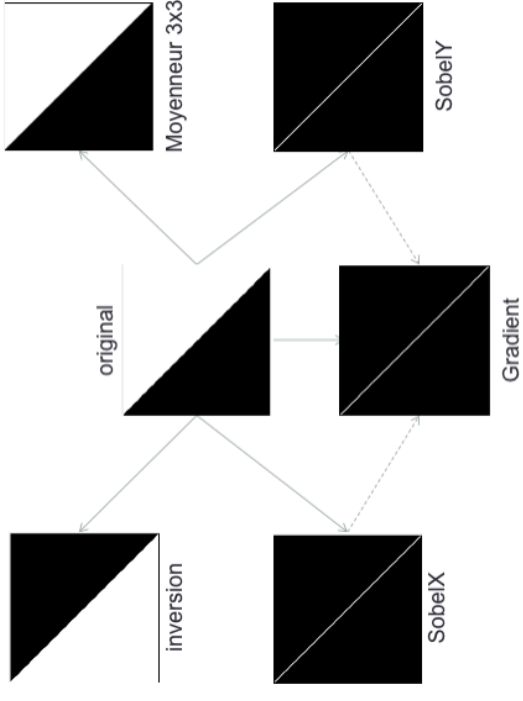
A partir d'un tableau 2D vide → remplir le tableau



Putpixels → image.txt → txt2gdr → xv

Faire les mêmes traitements (inversion, moyenne, Gx, Gy, Gradient)





Package : entree_sortie_pkg

```
package entree_sortie_pkg is
type IMAGE2D is array (natural range <>, natural range <>) of integer;
type MEM_FIFO_LIGNE is array (natural range <>) of std_logic_vector (7 downto 0);

function get_pixels(entree : string; N_col : integer; N_ligne : integer) return IMAGE2D;
procedure put_pixels(sortie : string; IM2D : IMAGE2D; n_pix_x : integer; n_pix_y : integer;
                    n_bit_pixel : integer);
end entree_sortie_pkg;
```

La fonction Put_Pixel

```
function get_pixels(entree : string; N_col : integer; N_ligne : integer) return IMAGE2D is
variable ligne : LINE;
file FI : TEXT is in entree;
variable IM2D : IMAGE2D(1 to N_ligne, 1 to N_col);
variable I : natural:=0;
begin
-- enlever l'entête
READLINE(FI,ligne); READLINE(FI,ligne); READLINE(FI,ligne); READLINE(FI,ligne); READLINE(FI,ligne);
loop
if not ENDFILE(FI) then
READLINE(FI,ligne);
I := I + 1;
else return IM2D;
end if;

for j in 1 to N_col loop
READ(ligne, IM2D(I,J));
end loop;

end loop;
end get_pixels;
```

La procédure Put_Pixel

```
procedure put_pixels(sortie : string; IM2D : IMAGE2D; n_pix_x : integer;
                    n_pix_y : integer; n_bit_pixel : integer) is
variable ligne : LINE;
file FO : TEXT is out sortie;
Begin

WRITE (LIGNE, STRING'("Type=GDR 2.0"));WRITELINE (FO,LIGNE);
WRITE (LIGNE, STRING'("Nb.Pixel/Ligne="));WRITE (LIGNE,n_pix_x);WRITELINE (FO,LIGNE);
WRITE (LIGNE,STRING'("Nb.Lignes="));WRITE (LIGNE,n_pix_x);WRITELINE (FO,LIGNE);
WRITE (LIGNE, STRING'("Nb.Bits="));WRITE (LIGNE,n_bit_pixel);WRITELINE (FO,LIGNE);
WRITE (LIGNE, STRING'(" "));WRITELINE (FO,LIGNE);

for I in 1 to n_pix_y loop
for J in 1 to n_pix_x loop
WRITE (LIGNE, IM2D(I,J));
WRITE (LIGNE, STRING'(" ")); -- Separateur
end loop;
WRITELINE (FO,LIGNE);
end loop;
end put_pixels;
```

Exemple ENTREE-SORTIE FICHIER (5)

```
library ieee;
use work.entree_sortie_pkg.all;
-----
entity Traitement is
generic( N_col : natural := 256; N_ligne: natural:= 256);
end Traitement ;
-----
architecture arch_Inv_niv of Traitement is
begin
process
variable tab_image: IMAGE2D(1 to N_col, 1 to N_ligne);
variable tab_image_traitee: IMAGE2D( 1 to N_col, 1 to N_ligne);
begin
tab_image := get_pixels("bureau.txt", N_col, N_ligne);
L1:for i in 1 to N_ligne-1 loop
L2: for j in 1 to N_col-1 loop
tab_image_traitee(i,j) := 255- tab_image(i,j); -- traitement
end loop L2;
end loop L1;
put_pixels("bureau_inv.txt",tab_image_traitee, N_col, N_ligne, 8);
wait;
end process;
end arch_Inv_niv;
```

Pas besoin de testbench !